

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования**

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №6.

Выполнил: студент группы БПИ2401

Исламов Эмин Маратович

Проверил: Харрасов Камиль Раисович

Цель работы

Изучить работу с коллекциями Java и обобщениями (generics):

- научиться применять Map для подсчёта частот, сортировать результаты и выводить топ значений;
- реализовать собственный обобщённый стек Stack<T> на массиве;
- применить коллекции для учёта продаж: хранение списка/структуры данных, подсчёт сумм и анализ “самого популярного” товара.

Ход работы

1. Реализовать программу **TopWords** (Задание 1): чтение текстового файла, подсчёт слов через Map, сортировка и вывод топ-10.
2. Реализовать обобщённый класс **Stack** (Задание 2): методы push, pop, peek и проверка на переполнение/пустоту.
3. Реализовать программу **SalesApp** (Задание 3, вариант 1 — ArrayList): ввод продаж, вывод списка, подсчёт общей суммы, поиск самого популярного товара.

ЗАДАНИЕ 1: TopWords.java

```
import java.util.*;
import java.io.*;

public class TopWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String filePath = sc.nextLine().trim();

        Map<String, Integer> wordCount = new HashMap<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] words = line.toLowerCase().replaceAll("[^a-zA-я0-9]+", " ")
                    .split("\\s+");
                for (String word : words) {
                    if (!word.isEmpty()) {
```

```

        wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
    }
}

} catch (FileNotFoundException e) {
    System.err.println("File not found: " + e.getMessage());
    return;
} catch (IOException e) {
    System.err.println("Error reading file: " + e.getMessage());
    return;
}

List<Map.Entry<String, Integer>> sortedWords = new
ArrayList<>(wordCount.entrySet());
sortedWords.sort((a, b) -> Integer.compare(b.getValue(), a.getValue()));

int limit = Math.min(10, sortedWords.size());
for (int i = 0; i < limit; i++) {
    System.out.println(sortedWords.get(i).getKey());
}
}
}

```

ЗАДАНИЕ 2: Stack.java

```

public class Stack<T> {
    private T[] data;
    private int size;

    @SuppressWarnings("unchecked")
    public Stack(int capacity) {
        data = (T[]) new Object[capacity];
        size = 0;
    }

    public void push(T element) {
        if(size == data.length) {
            throw new IllegalStateException("Stack is full");
        }
        data[size++] = element;
    }

    public T pop() {
        if(size == 0) {
            throw new IllegalStateException("Stack is empty");
        }
    }
}

```

```

        T value = data[--size];
        data[size] = null; // Clear reference
        return value;
    }

    public T peek() {
        if(size == 0) {
            throw new IllegalStateException("Stack is empty");
        }
        return data[size - 1];
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public int size() {
        return size;
    }
}

```

StackUsage.java

```

public class StackUsage {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>(5);

        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println("Top element: " + stack.peek()); // Should print 30

        while (!stack.isEmpty()) {
            System.out.println("Popped element: " + stack.pop());
        }
    }
}

```

ЗАДАНИЕ 1, вариант 1 - ArrayList: SalesApp.java

```

import java.util.*;

public class SalesApp {
    static class Sale {

```

```

private String product;
private int quantity;
private double price;

public Sale(String product, int quantity, double price){
    this.product = product;
    this.quantity = quantity;
    this.price = price;
}

public String getProduct(){
    return product;
}
public int getQuantity(){
    return quantity;
}
public double getPrice(){
    return price;
}
public double getTotal(){
    return quantity * price;
}

@Override
public String toString(){
    return "Product: " + product + ", Quantity: " + quantity + ", Price: " +
price + ", Total: " + getTotal();
}
}

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);
    ArrayList<Sale> sales = new ArrayList<>();

    while(true){
        System.out.println("Учет продаж на arraylist\n 1. Добавить продажу\n 2.
Показать все продажи\n 3. Показать общую сумму продаж\n 4. Самый популярный товар\n 0.
Выход\n Выбор:");
        int choice = sc.nextInt();
        sc.nextLine();
        if(choice == 0){
            break;
        }
        switch(choice){
            case 1:
                System.out.print("Название товара: ");
                String product = sc.nextLine().trim();
                System.out.print("Количество: ");

```

```

        int qty = sc.nextInt();
        System.out.print("Цена за единицу: ");
        double price = sc.nextDouble();
        sales.add(new Sale(product, qty, price));
        break;
    case 2:
        if (sales.isEmpty()) {
            System.out.println("Нет записей о продажах.");
        } else {
            for (Sale s : sales) {
                System.out.println(s.toString());
            }
        }
        break;
    case 3:
        double total = 0;
        for (Sale s : sales) {
            total += s.getTotal();
        }
        System.out.println("Общая сумма продаж: " + total);
        break;
    case 4:
        if (sales.isEmpty()) {
            System.out.println("Нет записей о продажах.");
        } else {
            Map<String, Integer> productCount = new HashMap<>();
            for (Sale s : sales) {
                productCount.put(s.getProduct().toLowerCase(),
productCount.getDefault(s.getProduct().toLowerCase(), 0) + s.getQuantity());
            }
            String bestProduct = "";
            int maxCount = 0;
            for (Map.Entry<String, Integer> entry :
productCount.entrySet()) {
                if (entry.getValue() > maxCount) {
                    maxCount = entry.getValue();
                    bestProduct = entry.getKey();
                }
            }
            System.out.println("Самый популярный товар: " + bestProduct +
" (Продано: " + maxCount + ")");
        }
        break;
    default:
        System.out.println("Неверный выбор. Попробуйте снова.");
    }
}
}

```

```
}
```

TopWords

- `HashMap<String,Integer>` хранит частоты слов.
- Текст нормализуется: `toLowerCase()` и `replaceAll("[^a-za-я0-9]+", " ")`.
- Потом `split("\\s+")` → получаем слова.
- Сортируем `entrySet()` по убыванию значения и печатаем первые 10.

Stack

- Внутри массив `T[] data`, индекс `size` показывает количество элементов.
- `push` кладёт элемент в `data[size]` и увеличивает `size`.
- `pop` уменьшает `size` и возвращает элемент.
- `peek` смотрит вершину без удаления.

SalesApp

- Храним продажи в `ArrayList<Sale>`.
- Счёт общей суммы через цикл.
- Самый популярный товар — через `Map<String,Integer>` по суммарному количеству.

Вывод

В ходе лабораторной работы были освоены основные коллекции Java (`Map`, `List`) и подходы к их применению: подсчёт частот, сортировка данных, хранение и анализ объектов. Также реализован обобщённый стек `Stack<T>` на массиве, что закрепило понимание `generics` и принципов работы структур данных.

Контрольные вопросы (ответы)

1. Какие интерфейсы коллекций есть в Java?

Iterable, Collection, List, Set, Queue, Deque, Map, SortedSet, NavigableSet, SortedMap, NavigableMap.

2. Какие классы коллекций есть в Java?

ArrayList, LinkedList, HashSet, TreeSet, LinkedHashSet, PriorityQueue, ArrayDeque, HashMap, TreeMap, LinkedHashMap, Hashtable, ConcurrentHashMap и др.

3. Что такое итератор?

Объект для последовательного обхода коллекции (hasNext(), next()), без знания внутреннего устройства коллекции.

4. Как работают коллекции на основе Map?

Хранят пары ключ-значение, ключ уникален. Быстрый доступ к значению по ключу (особенно в HashMap).

5. Как работают коллекции на основе List?

Упорядоченный список, допускает дубликаты, доступ по индексу.

6. Как работают коллекции на основе Set?

Хранят только уникальные элементы (без дубликатов).

7. Как можно синхронизировать коллекции в Java?

Collections.synchronizedList/map/set(...), либо использовать concurrent-коллекции (ConcurrentHashMap, CopyOnWriteArrayList).

8. Какие методы предоставляет интерфейс Collection?

add, remove, contains, size, isEmpty, clear, iterator, addAll, removeAll, retainAll, toArray и др.

9. Какие реализации интерфейса List вы знаете?

ArrayList, LinkedList, Vector (устарев.), CopyOnWriteArrayList.

10. Какие реализации интерфейса Set вы знаете?

HashSet, TreeSet, LinkedHashSet.

11. Что такое Comparable и Comparator?

Comparable задаёт “естественный порядок” внутри класса (compareTo).

Comparator задаёт внешний способ сравнения (compare), можно создавать разные сортировки.

12. Что такое параметр типа?

Это T, K, V в дженериках — placeholder, который заменится реальным типом при использовании.

13. Как параметризовать метод, класс?

Класс: `class Box<T> { ... }`

Метод: `static <T> T first(List<T> list) { ... }`

14. Что такое стирание типов (type erasure)?

Во время компиляции информация о тип-параметрах удаляется, и код превращается в работу с Object + вставки cast.

15. Как можно обойти ограничения стирания типов?

Передавать Class<T> в конструктор/метод, использовать фабрики, не создавать new T[], а создавать Object[] и приводить.

16. Как работают дженерики с массивами?

Нельзя создать new T[10] напрямую. Обычно создают Object[] и приводят к T[].

17. Можно ли создать массив дженериков?

Нельзя напрямую: new List<String>[10] запрещено.

18. Что такое wildcard тип?

? — неизвестный тип: List<?>, ограничения ? extends Number или ? super Integer.

19. Разница между и ?

? extends T — можно читать как T (producer), но нельзя безопасно добавлять.

? super T — можно безопасно добавлять T (consumer), читать только как Object.